That final advantage—better test output—bears a closer look. The original snippet would produce failure messages like this:

```
expected: 0
     got: 2

(compared using ==)

expected: 10000
     got: 9900

(compared using ==)
```

The output just reads "expected *x* / got *y*," with no hint about the higher-level concepts. By contrast, check out the failure messages from the second snippet:

```
expected #<Event "Art Show" (capacity: 100)> to have no tickets sold, but ↩
had 2

expected #<Event "U2 Concert" (capacity: 10000)> to be sold out, but had  ↩
100 unsold tickets
```

Not only does this report speak our domain language, it also provides additional details—such as what specific events we are testing here. In this chapter, we're going to show you how to build custom matchers like these by taking tiny steps forward from the RSpec concepts you already know.

Delegating to Existing Matchers Using Helper Methods

We're going to start with a technique you've already used to keep your code organized: helper methods. RSpec provides its own matchers via built-in methods, such as contain_exactly(...). You can easily write your own methods that return the same matcher objects but using names specific to your domain. You can also add your own customizations, such as default arguments.

When you developed the expense tracker application in *Building an App With RSpec 3*, you used the matcher expression a_hash_including(id: some_id) to represent a particular expected expense. Here's an example use:

12-creating-custom-matchers/02/expense_tracker/spec/integration/app/ledger_spec.rb

```
expect(ledger.expenses_on('2017-06-10')).to contain_exactly(
  a_hash_including(id: result_1.expense_id),
  a_hash_including(id: result_2.expense_id)
)
```

This matcher got the job done. Notice, though, how it expresses the expectation in terms of Ruby objects: *hashes* and *IDs*. The object you're describing is an *expense*. It would be nice to use the domain language of the application: